MediFlow

Especificação Funcional

VERSÃO	1.0 — escopo obrigatório e diferenciais esclarecidos
DATA	18/10/2026
BASE DOCUMENTAL	Projeto Oracle / Hackathon ONE G10 e MediFlow — Análise de Requisitos v1.1
ESTADO	decisões funcionais do solicitante incorporadas; permanecem definições operacionais e regras clínicas que a equipe deve registrar antes da implementação correspondente.

Documento funcional do MVP

Sumário

1. Objetivo e resultado esperado . . . 3
2. Fontes e precedência . . . 3
3. Escopo do MVP . . . 3
4. Atores e permissões funcionais . . . 4
5. Vocabulário e valores controlados . . . 4
6. Fluxo funcional ponta a ponta . . . 5
7. Requisitos funcionais detalhados . . . 6
8. Regras de negócio e precedência . . . 9
9. Modelo de dados funcional . . . 10
10. API REST proposta . . . 11
11. Requisitos de interface . . . 11
12. Erros, exceções e recuperação . . . 12
13. Armazenamento, histórico e rastreabilidade . . . 12
14. Requisitos não funcionais convertidos em verificação . . . 13
15. Decisões confirmadas e definições restantes . . . 13
16. Critérios de aceite do MVP . . . 14
17. Matriz de rastreabilidade resumida . . . 15
18. Referências usadas . . . 15

1. Objetivo e resultado esperado

O MediFlow recebe documentos clínicos ou administrativos, classifica o documento e sua prioridade, extrai os dados previstos, avalia qualidade e conflitos, decide o destino, encaminha para fila humana quando necessário, persiste documento e resultado no OCI Object Storage e devolve resposta estruturada. O MVP demonstra pelo menos os fluxos padrão, urgente e ambíguo.

A solução é apoio à triagem documental. Não substitui diagnóstico, prescrição, decisão clínica ou auditoria profissional. Uma sugestão produzida pelo modelo deve ser identificável como tal e não pode ser apresentada como dado literal extraído do documento.

2. Fontes e precedência

1. Projeto_oracle.pdf: define objetivo, escopo do hackathon, capacidades e entregáveis mínimos.
2. MediFlow_-_Análise_de_Requisitos_v1.1.pdf: organiza o entendimento da equipe, propõe requisitos e registra decisões tomadas e pendentes.
3. Esta especificação: detalha comportamento para implementação. Definições não confirmadas aparecem como pendentes e não devem ser tratadas como decisões aprovadas.

Quando a análise acrescenta uma restrição ou decisão que não está no brief, ela é identificada como proposta da equipe, não como exigência original do Projeto Oracle.

3. Escopo do MVP

3.1 Incluído

- Receber documento por API REST e pela interface React.
- Aceitar conteúdo textual e arquivos PDF e de imagem; receber metadados JSON na chamada da API.
- Validar requisição, tipo, tamanho configurado e campos obrigatórios.
- Interpretar texto ou imagem/PDF com o modelo multimodal Gemini, por camada de integração isolada.
- Classificar tipo documental, especialidade e prioridade.
- Extrair os dados definidos nesta especificação, preservando ausência e incerteza.
- Verificar consistência, risco/urgência e confiança; aplicar fluxo condicional com LangGraph.
- Encaminhar para destinos lógicos simulados e fila de revisão humana; gerar evento de notificação para urgências.
- Persistir arquivo original e resultado no OCI Object Storage e registrar o histórico de triagem em JSON.
- Expor API e interface de triagem com status, classificação, destino, justificativa e histórico autorizado.
- Demonstrar os três cenários obrigatórios com dados fictícios: fluxo padrão, caso urgente e caso ambíguo ou inconsistente encaminhado para revisão humana.

3.2 Itens não obrigatórios no MVP

Esta seção não lista tarefas que a equipe precisa implementar. Ela separa diferenciais opcionais e integrações externas que o Projeto Oracle não exige para o MVP. O roteamento funcional continua obrigatório: o MediFlow deve escolher e registrar o destino lógico correto. No MVP, esse destino pode ser uma fila simulada, sem conexão com o sistema hospitalar de destino.

3.2.1 Diferenciais opcionais citados no Projeto Oracle

Podem ser considerados depois do núcleo obrigatório; não são critérios obrigatórios de aceite do MVP:

- Deploy na OCI Compute: hospedagem da aplicação ou de um contêiner n8n em máquina virtual Always Free da OCI com acesso público.
- OCR/visão especializada para receitas manuscritas ou outros documentos escaneados difíceis.

- Painel HITL dedicado em Streamlit para auditor aprovar/rejeitar documentos. Para este MVP, a fila é lógica, sem decisão humana interativa.
- Configuração de regras, destinos e limiar pelo gestor, incluindo slider em Streamlit.
- Alertas em tempo real por fluxo n8n, e-mail ou Slack. Para este MVP, registra-se apenas um evento de alerta simulado.

3.2.2 Integrações externas não exigidas pelo brief

O Projeto Oracle exige lógica de roteamento, apresentação por interface/API e persistência no OCI Object Storage, mas não especifica conexão ativa com prontuário eletrônico, farmácia, operadora, laboratório ou outros sistemas hospitalares. Portanto, essas conexões reais estão fora do escopo obrigatório do MVP. A menção a guias TISS/TUSS no contexto do setor descreve documentos/domínio; não exige integração com serviços TISS/TUSS.

E-mail e Slack aparecem apenas como exemplos de canais em um diferencial opcional de alertas em tempo real. A integração com esses serviços também não é obrigatória.

3.2.3 Limites e decisões específicas do MediFlow

- O uso de dados exclusivamente fictícios durante todo o MVP é uma decisão confirmada pelo solicitante. O brief exige documentos fictícios em testes, mas não formula essa restrição para todas as etapas do MVP.
- Diagnóstico clínico autônomo e prescrição não fazem parte das funções descritas para o MediFlow; o sistema classifica, extrai, sinaliza e roteia documentos para revisão/fluxos de destino.

4. Atores e permissões funcionais

Ator	Ações no MVP
Usuário remetente	Enviar documento e consultar o resultado da própria requisição conforme a interface/API disponível.
Operador de triagem	Consultar documentos e resultados apresentados pela interface; reprocessamento manual, se habilitado, deve gerar novo evento de histórico.
Auditor humano	Fora do MVP. Os casos são identificados e disponibilizados em fila lógica; aprovar, rejeitar ou corrigir dados será tratado em versão futura.
Administrador técnico	Configurar variáveis de ambiente e integração OCI/modelo; não altera limiar em tela no MVP.
Sistema MediFlow	Validar, classificar, extrair, avaliar, rotear, persistir, registrar histórico e produzir alertas/eventos.

Definição pendente da equipe: autenticação, autorização por papel e escopo, e identificação do auditor não estão detalhados nos documentos-base. Definir o mecanismo e os limites de acesso antes de expor a aplicação fora da demonstração local.

5. Vocabulário e valores controlados

5.1 Tipo de documento

Valores mínimos sugeridos, correspondentes ao projeto: `RECEITUARIO_MEDICO`, `LAUDO_EXAME_LABORATORIAL`, `LAUDO_EXAME_IMAGEM`, `GUIA_PROCEDIMENTO`, `RELATORIO_ALTA_INTERNACAO`, `ATESTADO_MEDICO`, `OUTRO`, `NAO_IDENTIFICADO`.

A especificação mantém laboratorial e imagem separados para permitir roteamento/demonstração; a análise original os agrupou em alguns trechos. A classificação deve guardar o rótulo apresentado ao usuário e o código estável.

5.2 Prioridade

MVP: `URGENTE` e `ROTINA`; `NAO_IDENTIFICADA` é permitido quando a informação não puder ser determinada, caso que deve ser encaminhado à revisão. A regra exata para prioridades intermediárias fica fora do MVP até decisão explícita.

5.3 Status de processamento

`RECEBIDO`, `PROCESSANDO`, `PROCESSADO`, `EM_REVISAO`, `ERRO_PROCESSAMENTO`, `ERRO_PERSISTENCIA`.

Status	Significado
`RECEBIDO`	O MediFlow aceitou a requisição e atribuiu um ID. O processamento ainda não começou.
`PROCESSANDO`	O documento está sendo analisado: leitura, classificação, extração e decisão de roteamento.
`PROCESSADO`	A análise terminou e não há motivo para revisão humana. O resultado foi salvo.
`EM_REVISAO`	A análise terminou, mas encontrou ambiguidade, conflito, score abaixo do limite ou ausência de um campo obrigatório. O caso entra na fila lógica; no MVP, um auditor não aprova nem corrige o documento pela interface.
`ERRO_PROCESSAMENTO`	A análise não pôde ser concluída, por exemplo, por falha do modelo ou por uma resposta inválida.
`ERRO_PERSISTENCIA`	O MediFlow não conseguiu salvar o documento ou seu resultado no OCI; portanto, não confirma que a triagem foi concluída com sucesso.

5.4 Destinos lógicos

`FILA_URGENCIA_MEDICA`, `FARMACIA_HOSPITALAR`, `AUDITORIA_AUTORIZACOES`, `PRONTUARIO_ELETRONICO`, `FILA_REVISAO_HUMANA`.

No MVP, são destinos lógicos/simulados. A resposta deve indicar que não houve integração com o sistema externo, quando aplicável.

6. Fluxo funcional ponta a ponta

1. O cliente envia arquivo ou texto e metadados. A API valida formato, campos e limites configurados.
2. O sistema gera ou valida `document_id`, registra o recebimento e persiste o original no bucket/caminho de recebidos. Se essa persistência obrigatória falhar, não informa sucesso de triagem.
3. O sistema obtém o conteúdo para análise. Para PDF/imagem, envia conteúdo suportado ao pipeline multimodal; para texto, usa o texto recebido. Falha de leitura gera motivo explícito e rota de revisão ou erro conforme seção 12.
4. O modelo devolve classificação e dados estruturados conforme esquema. A camada de aplicação valida o esquema com Pydantic e rejeita ou recupera respostas malformadas por política limitada e documentada.
5. O sistema calcula/recebe scores separados, registra evidências e avalia campos ausentes, conflitos, sinais urgentes e alto risco.
6. O grafo aplica as regras de negócio em ordem determinística. A saída inclui decisão, destino, motivo e indicação de revisão/notificação.
7. O sistema persiste resultado e evento/histórico no OCI. Um documento roteado a revisão deve ser armazenado no estado/caminho de auditoria; outros resultados seguem seu estado correspondente.

8. A API responde com status, dados, decisão e localização lógica de armazenamento, sem expor credenciais internas.
9. A interface exibe o resultado e distingue claramente rotina, urgência, revisão e falha.

7. Requisitos funcionais detalhados

RF-01 — Receber documento

O sistema deve permitir envio por endpoint REST e pela tela React. Cada requisição deve conter um arquivo ou conteúdo textual, nunca ambos vazios. Deve suportar metadados JSON na requisição. Uma requisição inválida recebe erro legível e não inicia inferência.

RF-02 — Formatos de entrada

Formatos do MVP: PDF, imagem raster comum (lista final de extensões/MIME a configurar) e texto. O conteúdo JSON é o envelope da API e pode conter texto/metadados; não significa que qualquer JSON arbitrário seja um documento clínico. PDF digitalizado e imagem passam pelo pipeline multimodal disponível. OCR especializado de manuscrito é diferencial e não pode ser prometido como capacidade garantida.

RF-03 — Validar entrada

Validar presença e tipo dos campos, arquivo não vazio, tipo de mídia permitido, tamanho máximo configurado e consistência entre extensão e MIME. Rejeitar com código e mensagem sem devolver stack trace ou segredo. Limites máximos precisam ser configurados antes da implementação.

RF-04 — Ler conteúdo

O sistema deve encaminhar texto, PDF e imagem ao adaptador Gemini, usando apenas o conteúdo necessário ao processamento. Se o provedor não responder, atingir timeout ou retornar conteúdo inválido, aplicar o tratamento de falhas da seção 12.

RF-05 — Classificar tipo

Identificar um dos tipos controlados da seção 5.1. Se não houver evidência suficiente, usar `NAO_IDENTIFICADO` e exigir revisão.

RF-06 — Classificar especialidade

Retornar especialidade(s) identificada(s) como texto estruturado. Quando não constar ou não puder ser inferida com segurança, retornar `null` e sinalizar ausência; não inventar especialidade.

RF-07 — Classificar prioridade

Identificar `URGENTE`, `ROTINA` ou `NAO_IDENTIFICADA` com score/evidência. Sinais críticos/urgentes devem disparar regra de urgência mesmo com confiança geral baixa. Se também houver ambiguidade, ausência de campo obrigatório, conflito impeditivo ou score baixo, revisão humana também é obrigatória.

RF-08 — Extrair paciente

Extrair nome e idade quando explícitos. Campo ausente permanece `null`; nome/idade não podem ser completados por inferência. Registrar a fonte/trecho quando disponível e seguro para auditoria.

RF-09 — Extrair profissional

Extrair nome e CRM do solicitante quando presentes. Armazenar CRM como cadeia, preservando número e UF normalizados em campos separados (`numero`, `uf`) e, se útil, valor original. Não validar conselho contra base externa no MVP.

RF-10 — Extrair diagnóstico e CID-10

Extrair hipótese/diagnóstico principal e CID explicitamente presente. Distinguir `cid_extraido` de acordo com o documento de CIDs fornecido para consulta. Até decisão da seção 15, não sugerir CID ausente nem apresentar inferência como extração. Se não houver CID, usar `null` e sinalizar ausência.

RF-11 — Extrair medicamentos e doses

Retornar lista, possivelmente vazia, com nome, concentração, dose, via, frequência e duração quando cada item estiver explicitamente disponível. Cada atributo ausente fica `null`. A lista vazia deve distinguir “nenhum identificado” de “leitura falhou” por meio de sinalizadores de extração. A medicação será comparada com a farmacopeia para validar nome e dosagem da medicação.

RF-12 — Extrair exames

Retornar exames solicitados e exames realizados em listas separadas, com nome e detalhes explícitos. Não deduzir exames a partir do diagnóstico.

RF-13 — Identificar urgência

Retornar indicador booleano/estado (`sim`, `nao`, `indeterminado`), sinais encontrados, motivo e referência ao conteúdo. Não declarar `nao` se o documento estiver ilegível ou a análise não tiver sido concluída; nesse caso usar `indeterminado`.

RF-14 — Validar saída estruturada

Validar todo resultado por modelos Pydantic versionados. Campo desconhecido não deve quebrar o consumidor; campos obrigatórios do envelope, enumerações e tipos devem ser validados. Falha de validação deve acionar tratamento de erro/fallback e ficar registrada.

RF-15 — Preservar ausência e origem

Não preencher dado clínico ausente. Cada campo extraído deve permitir distinguir valor explícito, ausente e incerto. Usar `null` para ausência, mais `campos_ausentes` e `campos_incerto`; guardar trecho de evidência quando existir. Campos ausentes que forem obrigatórios conforme RF-21 geram revisão humana. Não guardar explicação interna do modelo como se fosse prova.

RF-16 — Avaliar consistência e conflitos

Executar verificações determinísticas previstas e registrar conflito, campos envolvidos, gravidade e justificativa. No MVP, exemplos incluem conflito explícito entre CID e diagnóstico quando há regra definida, dose suspeita sinalizada (sem alterar dose), campos contraditórios e discrepância entre prioridade e sinais textuais. Regras clínicas específicas precisam ser aprovadas/fornecidas pela equipe; o sistema não deve inventar tabela clínica.

RF-17 — Calcular confiança de classificação

Retornar `score_confianca_classificacao` entre 0 e 1. A origem do score e limitações devem ser documentadas; score autorrelatado pelo LLM não deve ser tratado como probabilidade calibrada sem validação.

RF-18 — Calcular confiança de extração e combinada

Retornar scores de extração e combinado no intervalo [0,1], com fórmula configurável/documentada. O limiar padrão é `0.85`, vindo de variável de configuração. A fórmula aprovada é `min(score_confianca_classificacao, score_extracao)`. Comparação: combinado `< limiar` envia para revisão; combinado `>= limiar` apenas permite fluxo automático se nenhuma outra regra exigir revisão/urgência. O limiar padrão é `0.85`.

RF-19 — Orquestrar decisão

Executar nós condicionais no LangGraph ou mecanismo equivalente aprovado, produzindo resultado reprodutível a partir dos mesmos sinais/regras. A decisão registra versão do grafo/regras e timestamps relevantes.

RF-20 — Rotear

Selecionar destino lógico segundo tipo, prioridade, sinais de risco, ambiguidade, confiança e regras da seção 8. O destino não equivale a transmissão para sistema externo no MVP.

RF-21 — Revisão humana

Enviar à `FILA_REVISAO_HUMANA` se houver ambiguidade, ilegibilidade, conflito impeditivo, score combinado abaixo do limiar ou ausência de qualquer campo obrigatório: nome do paciente, idade, nome do médico solicitante, CRM ou diagnóstico/hipótese diagnóstica. CID-10, medicamentos e exames ausentes não acionam esta regra por si sós. Caso também exista urgência, encaminhar simultaneamente à fila urgente e registrar evento de alerta simulado. A fila é lógica; aprovar/rejeitar/corrigir por auditor humano está fora do MVP. A decisão deve informar código/motivo da revisão.

RF-22 — Notificar urgência

Gerar evento de notificação para casos urgentes/críticos, com identificador, prioridade, destino/canal lógico, motivo, timestamp e resultado de geração. No MVP, registrar evento de alerta simulado no resultado/histórico e exibi-lo na interface/log. Não enviar e-mail, Slack ou outro alerta externo e não afirmar entrega externa.

RF-23 — Justificar decisões

Toda decisão inclui justificativa estruturada, códigos de regra acionados e sinais considerados. Deve ser concisa, rastreável e sem alegar certeza clínica não suportada.

RF-24 — Fallback e exceções

Falha de leitura, modelo, validação, grafo, notificação ou storage gera status/código apropriado. Para risco ou incerteza clínica, fallback é revisão humana; se a persistência não estiver disponível, reter estado de erro e não afirmar documento armazenado. Detalhes e retries na seção 12.

RF-25 — Responder processamento

Retornar status, documento_id, classificação, dados extraídos, scores, conflitos, decisão, notificação e armazenamento. Em erros, retornar envelope consistente com código, mensagem segura, etapa e possibilidade de reenvio.

RF-26 — Consultar histórico

Disponibilizar histórico de triagem em JSON por documento_id, com eventos ordenados temporalmente e restritos a chamadas autorizadas. O mecanismo de consulta é API; a rota proposta está na seção 10.

RF-27 — Interface e API

A tela React permite envio, apresentação de processamento e exibição do resultado e erros. A API FastAPI fornece submissão e consulta. A tela não deve expor chaves, stack traces, prompt interno ou conteúdo de outro documento.

RF-28 — Persistir no OCI

Guardar original e resultado no OCI Object Storage usando recursos Always Free. Buckets/nomes são configuração do ambiente, não hard-coded. Segredos OCI ficam fora do repositório.

RF-29 — Organizar por estado

Segregar objetos/caminhos por estado funcional, no mínimo recebidos, processados e auditoria humana. A decisão entre buckets distintos e prefixos em bucket único é de implantação; preservar segregação lógica e política de acesso.

RF-30 — Informar armazenamento

Retornar bucket lógico, caminho/identificador do objeto e status de persistência. Não devolver URL pública ou credencial. Se persistência de entrada ou saída falhar, informar falha e não marcar sucesso completo.

8. Regras de negócio e precedência

8.1 Regras aprovadas pelo material-base

- `RN-01`: sem ambiguidade/conflito impeditivo ou score abaixo do limiar, encaminhar automaticamente ao destino de rotina do tipo; com urgência, à fila urgente e com alerta.
- `RN-02`: documento ambíguo, ilegível, inconsistente ou com score combinado abaixo do limiar vai para revisão humana.
- `RN-03`: achado crítico ou prescrição de alto risco deve gerar encaminhamento para fila de urgência e alerta.
- `RN-04`: persistir documento e decisão com estado correspondente.
- `RN-05`: registrar justificativa estruturada para toda decisão.
- `RN-06`: usar somente recursos Always Free OCI.
- `RN-07`: cada documento possui `documento_id` único gerado pelo sistema quando não informado/aceito pelo contrato.

8.2 Tabela de decisão proposta

Sinal	Destino lógico	Revisão	Notificação
Urgência/achado crítico	`FILA_URGENCIA_MEDICA`	Não	Sim, evento obrigatório
Score combinado abaixo de 0,85	`FILA_REVISAO_HUMANA`	Sim	Não, salvo urgência simultânea
Urgência associada a ambiguidade, ilegibilidade ou conflito impeditivo	`FILA_URGENCIA_MEDICA` e `FILA_REVISAO_HUMANA`	Sim	Sim, evento de alerta
Receita de rotina, sem impedimento	`FARMACIA_HOSPITALAR`	Não	Não
Guia de procedimento de rotina	`AUDITORIA_AUTORIZACOES`	Não	Não
Laudo, alta ou atestado de rotina	`PRONTUARIO_ELETRONICO`	Não	Não
Tipo desconhecido ou destino não mapeado	`FILA_REVISAO_HUMANA`	Sim	Só se também urgente

Urgência e alto risco sempre geram encaminhamento à fila urgente e alerta. Quando coexistirem com ambiguidade, dado obrigatório ausente, conflito impeditivo ou score baixo, a revisão humana também é obrigatória. O payload registra os dois destinos/ações, sem reduzir a decisão a um destino único.

9. Modelo de dados funcional

9.1 Requisição de triagem

```
{
  "documento_id": null,
  "tipo_arquivo": "PDF",
  "nome_arquivo": "laudo.pdf",
  "documento_texto": null,
  "canal_origem": "Pronto_Socorro",
  "metadados": {}
}
```

Enviar arquivo em multipart/form-data ou texto JSON conforme a rota. `documento_id` é opcional na entrada; quando ausente, é gerado. Não duplicar arquivo em base64 dentro do JSON sem decisão de contrato.

9.2 Resposta de triagem

```
{
  "status": "EM_REVISAO",
  "documento_id": "DOC-...",
  "classificacao": {
    "tipo_documento": "LAUDO_EXAME_IMAGEM",
    "especialidade": ["Radiologia"],
    "nivel_prioridade": "URGENTE",
    "score_confianca_classificacao": 0.99
  },
  "dados_extraidos": {
    "paciente": {"nome": "...", "idade": 52},
    "medico_solicitante": {"nome": "...", "crm": {"numero": "145892", "uf": "SP"}},
    "diagnostico_principal": "...",
    "cid_extraido": null,
    "medicamentos": [],
    "exames_solicitados": [],
    "exames_realizados": ["..."],
    "urgencia": {"estado": "SIM", "sinais": ["..."], "justificativa": "..."}
  },
  "qualidade": {
    "score_extracao": 0.97,
    "score_combinado": 0.97,
    "limiar_aplicado": 0.85,
    "campos_ausentes": ["cid_extraido"],
    "campos_incerto": [],
    "conflitos": []
  },
  "decisao_roteamento": {
    "destinos": ["FILA_URGENCIA_MEDICA", "FILA_REVISAO_HUMANA"],
    "requer_auditoria_humana": true,
    "justificativa": "...",
    "regras_acionadas": ["RN-03", "RN-02"],
    "notificacao": {"gerada": true, "canal": "SIMULADO", "status": "REGISTRADA"}
  },
  "armazenamento_oci": {
    "bucket": "mediflow-documentos-clinicos",
    "caminho_objeto": "auditoria/DOC-.../resultado.json",
    "status": "SUCESSO"
  },
  "criado_em": "ISO-8601",
  "versao_esquema": "1.0"
}
```

Exemplo ilustrativo de contrato, não define dados reais nem confirma decisões pendentes. Valores clínicos devem vir do documento fictício da demonstração. O exemplo do projeto usa `cid_sugerido`; este contrato evita misturar sugestão com extração até validação.

9.3 Histórico

Cada evento deve conter `evento_id`, `documento_id`, `tipo_evento`, `criado_em`, `ator` (sistema/usuário), `status_anterior`, `status_novo`, `motivo`, `versao_esquema` e correlação da requisição. Correções humanas devem preservar valor anterior, valor corrigido, autor e justificativa; não sobrescrever silenciosamente o resultado original.

10. API REST proposta

Rotas abaixo são proposta de contrato funcional, sujeita à convenção de implementação:

Método e rota	Uso	Resposta esperada
`POST /api/v1/triagens`	Enviar documento para triagem. multipart ou JSON textual conforme content type.	`202 Accepted` com `documento_id`, status `RECEBIDO`/`PROCESSANDO` e referência de consulta. O processamento é assíncrono.
`GET /api/v1/triagens/{documento_id}`	Consultar estado e resultado atual.	`200 OK`; `404` se inexistente; `403` se não autorizado.
`GET /api/v1/triagens/{documento_id}/historico`	Consultar eventos de histórico.	`200 OK` com lista ordenada.
`GET /api/v1/health`	Verificar disponibilidade técnica sem expor segredos.	`200`/`503`, sem dado clínico.

Exemplo de resposta de submissão aceita:

```
{
  "documento_id": "DOC-CLIN-2026-0001",
  "status": "RECEBIDO",
  "status_url": "/api/v1/triagens/DOC-CLIN-2026-0001",
  "historico_url": "/api/v1/triagens/DOC-CLIN-2026-0001/historico"
}
```

A consulta pode retornar `RECEBIDO`, `PROCESSANDO`, `PROCESSADO`, `EM_REVISAO`, `ERRO_PROCESSAMENTO` ou `ERRO_PERSISTENCIA`. `PROCESSADO` e `EM_REVISAO` são estados terminais da automação para a requisição atual; a fila não inclui decisão humana interativa no MVP.

10.1 Erros HTTP propostos

HTTP	Código funcional	Quando
400	`REQUISICAO_INVALIDA`	JSON malformado ou parâmetros incompatíveis.
401/403	`NAO_AUTORIZADO`	Quando autenticação/autorização for implementada.
404	`DOCUMENTO_NAO_ENCONTRADO`	Consulta a ID inexistente ou não visível.
413	`ARQUIVO_MAIOR_QUE_LIMITE`	Tamanho excede configuração.
415	`FORMATO_NAO_SUPORTADO`	Tipo de arquivo não aceito.
422	`CAMPOS_INVALIDOS`	Campos ausentes/inválidos na requisição.
429	`LIMITE_DE_REQUISICOES`	Se houver limitação configurada.
500	`ERRO_INTERNO`	Falha não recuperável, mensagem segura.
502/503/504	`PROVEDOR_INDISPONIVEL` / `SERVICO_INDISPONIVEL` / `TIMEOUT`	Dependência externa indisponível ou timeout.

O modo assíncrono é decisão aprovada: a submissão retorna `202 Accepted`; o cliente consulta o resultado por `documento_id` até status terminal.

11. Requisitos de interface

1. Tela de envio com seleção de arquivo ou entrada de texto, campo de origem e ação clara de enviar.
2. Validar tipo/tamanho no cliente para usabilidade e novamente na API para segurança.

3. Apresentar estados: aguardando, enviando, recebido, processando, processado, em revisão e erro.
4. Resultado deve exibir tipo, especialidade, prioridade, scores, dados extraídos, ausências/incertezas, conflitos, destino(s), justificativa, status de alerta e status de persistência.
5. Urgência deve ter destaque visual acessível e texto, não depender apenas de cor.
6. Não exibir dado como extraído quando for sugestão; mostrar ausências como “não identificado no documento”.
7. Erros devem indicar ação possível (corrigir arquivo, tentar novamente, consultar ID), sem stack trace, prompt ou segredo.
8. Histórico deve estar ordenado por data, com status e motivo.
9. Conteúdo da interface deve esclarecer que destinos e alertas são simulados no MVP. A fila de revisão deve listar/identificar os casos, sem fluxo de aprovação/rejeição humana.

12. Erros, exceções e recuperação

Situação	Comportamento funcional	Estado/resultado
Formato/campos inválidos	Rejeitar antes de chamar Gemini; explicar campo/limite.	Erro de requisição; sem triagem.
Documento ilegível ou extração inconclusiva	Marcar campos incertos; encaminhar para revisão, sem preencher lacunas.	`EM_REVISAO`.
Gemini indisponível/timeout	Retry limitado e configurado; se persistir, registrar falha e permitir reenvio/revisão.	`ERRO_PROCESSAMENTO` ou `EM_REVISAO` se a informação já indicar risco.
Resposta Gemini inválida para esquema	Validar; tentativa de reparo limitada se definida; não aceitar payload inválido silenciosamente.	Fallback revisão/erro.
Conflito entre dados	Registrar conflito sem alterar valores de origem; exigir revisão quando impeditivo.	`EM_REVISAO` conforme regra.
Evento urgente	Registrar evento de alerta simulado na resposta/histórico e exibi-lo na interface/log; não chamar serviço externo.	Destino urgente; `status=SIMULADO`.
Falha ao registrar evento de alerta simulado	Não apagar triagem; registrar falha interna e preservar encaminhamento urgente.	Triagem urgente; evento com falha rastreada.
OCI falha ao gravar original	Não declarar documento recebido com sucesso durável; retry limitado.	`ERRO_PERSISTENCIA`; resposta de erro.
OCI falha ao gravar resultado	Preservar/retry; não marcar sucesso completo.	`ERRO_PERSISTENCIA`; resultado não confirmado.
Erro interno	Registrar identificador de correlação e detalhe técnico restrito.	Mensagem externa segura.

Limites/retries/timeouts concretos são variáveis operacionais a definir e registrar; não devem ser arbitrariamente embutidos em regras clínicas.

13. Armazenamento, histórico e rastreabilidade

- Usar OCI Object Storage Always Free conforme o limite e a disponibilidade da conta/projeto.
- Proposta de organização por prefixo: `recebidos/{ano}/{mes}/{documento_id}/original`, `processados/.../resultado.json`, `auditoria/.../resultado.json`. A alternativa com buckets separados deve manter o mesmo significado funcional.
- Registrar, no mínimo, recebimento, início/fim de processamento, decisão, destino, alerta, persistência e eventual correção humana.

- Incluir identificadores e versão de esquema/regras/modelo; evitar armazenar segredos e dados desnecessários em logs.
- Definir política de retenção e exclusão antes de qualquer uso com dados reais. Os documentos-base não especificam retenção.
- O usuário recebe identificador/caminho lógico, não URL pública nem chave OCI.

14. Requisitos não funcionais convertidos em verificação

ID	Requisito	Verificação funcional proposta
RNF-01 Precisão	Erros clínicos são graves.	Casos de demonstração com resultado esperado aprovado pela equipe; métricas e conjunto de validação antes de alegar precisão. Meta numérica pendente.
RNF-02 Segurança clínica	Não encaminhar urgência como rotina; dúvida exige revisão.	Testar exemplos urgentes e ambíguos, incluindo sinais contraditórios e score baixo.
RNF-03 Confiabilidade	Falha não deve perder documento.	Simular indisponibilidade de Gemini/OCI; status e retry devem ser observáveis.
RNF-04 Rastreabilidade	Toda decisão tem justificativa/histórico.	Consultar histórico e verificar ator, motivo, status e versões.
RNF-05 Custo	Somente OCI Always Free.	Revisão da configuração de recursos e monitoramento do consumo.
RNF-06 Formato	Saída JSON válida tipada.	Validar resposta contra Pydantic/OpenAPI e versão declarada.
RNF-07 Agilidade	Resposta rápida para urgências.	Medir latência por etapa e estabelecer meta após primeiro pipeline; meta pendente.
RNF-08 Privacidade	Usar somente dados fictícios em todo o MVP, conforme decisão do solicitante.	Validar exemplos com dados sintéticos; manter segredo fora do Git.
RNF-09 Manutenibilidade	Código modular, commits e README.	Verificar README, diagrama e separação do adaptador LLM.
RNF-10 Flexibilidade	Modelo substituível.	Teste contratual do adaptador sem refazer regras de decisão.
RNF-11 Usabilidade	Mostrar tipo, prioridade, destino e justificativa.	Revisão dos três fluxos pela equipe.

15. Decisões confirmadas e definições restantes

Decisões confirmadas pelo solicitante e normativas para esta versão:

1. Urgência: encaminhar à fila urgente e registrar alerta simulado; com ambiguidade, incluir também na fila lógica de revisão. Urgência isolada termina como processada, sem revisão humana.
2. CID-10: extrair somente se explícito; ausência fica `null`, sem sugestão/inferência.
3. Campos cuja ausência exige revisão: nome do paciente, idade, nome do médico solicitante, CRM e diagnóstico/hipótese diagnóstica. A ausência de CID-10, medicamentos ou exames não exige revisão por si só, apenas a inconsistência.
4. Confiança combinada: mínimo entre score de classificação e score de extração; abaixo de `0.85` exige revisão.
5. Dados: somente fictícios em todo o MVP.
6. API: processamento assíncrono, `202 Accepted` e consulta posterior por ID.

7. Fila humana: fila lógica sem aprovação/rejeição/correção interativa no MVP.
8. Alertas: evento simulado sem integração externa.
9. Critérios clínicos: somente critérios explícitos, determinísticos e aprovados pela equipe; o modelo não define livremente risco alto ou conflito.

Definições restantes para a equipe antes da implementação correspondente:

- Limites de tamanho, páginas, extensões/MIME, timeout e política de retry.
- Formato final do upload multipart/JSON.
- Regras/dicionário aprovados para achado crítico, medicamento de alto risco e conflitos, incluindo fonte e responsável pela aprovação.
- Autenticação/autorização e identificação de quem pode consultar cada documento.
- Política de retenção e exclusão dos dados fictícios e dos históricos.
- Meta mensurável de latência e precisão, além dos casos de aceitação.

16. Critérios de aceite do MVP

1. Enviar texto e pelo menos um PDF e uma imagem aceitos, com resultado ou erro de validação coerente.
2. Documento padrão fictício: classificado, campos estruturados, sem sinal de urgência, score suficiente e destino lógico de rotina correto.
3. Documento urgente fictício: prioridade/sinal urgente detectado, destino de urgência e evento de alerta registrado; nunca encaminhado somente como rotina.
4. Documento ambíguo/ilegível fictício: revisão humana indicada, justificativa e campos incertos preservados, sem dado inventado.
5. Ausência de nome/idade do paciente, nome/CRM do solicitante ou diagnóstico: revisão obrigatória; ausência de CID-10 isoladamente não exige revisão.
6. Submissão da API retorna `202` com ID e consulta posterior mostra estados e resultado final.
7. Evento urgente aparece como simulado e não é enviado a serviço externo; a fila de revisão não permite decisão humana interativa no MVP.
8. Score abaixo do limiar padrão 0,85: revisão humana obrigatória; score igual ao limiar não é encaminhado por esse critério isoladamente.
9. Campos ausentes não são completados por inferência; aparecem como ausentes/incertos conforme schema.
10. Resultado e documento são persistidos no OCI e resposta informa status/caminho lógico; falha simulada não aparece como sucesso.
11. Consulta de resultado e histórico devolve eventos ordenados e JSON validado.
12. Regras de conflito, versão de configuração, justificativa e destino são rastreáveis.
13. Interface mostra estados, urgência, revisão, destino e falhas sem expor segredos.
14. README descreve arquitetura, grafo de decisão e instruções de execução; demonstração cobre os três cenários requeridos.
15. Nenhuma funcionalidade opcional é declarada entregue sem implementação demonstrável.

17. Matriz de rastreabilidade resumida

Fonte/requisito	Seções desta especificação
Brief: ingestão e formatos	3, 6, RF-01 a RF-04, 9, 10
Brief: classificação e extração clínica	RF-05 a RF-15, 9
Brief: consistência, score, urgência, grafo e revisão	RF-16 a RF-24, 8, 12
Brief: JSON, histórico e interface/API	RF-25 a RF-27, 9 a 11
Brief: OCI, Always Free, segregação	RF-28 a RF-30, 13
Brief: três cenários e entregáveis	3, 16
Análise v1.1: requisitos derivados/decisões	5, 8, 14, 15
Diferenciais (OCI Compute, OCR especializado, Streamlit, n8n)	3.2

18. Referências usadas

- `Projeto_oracle.pdf`
- `MediFlow_-_Análise_de_Requisitos_v1.1.pdf`

Ambos os documentos podem ser acessados através Google Drive: Abrir pasta de referência no Google Drive